Apache Tomcat Security Compliance Scanner
Application Server Configuration Audit Project

TABLE OF CONTENTS

Introduction

- Project Objectives 2
- Scope and Testing Environment 3
- Problem Statement & the Importance of Secure Configuration 3

Technical Implementation

- Documentation and Usability 4
- Scanner Architecture and Functionality 4-5

Demonstration and Results

- Environment Preparation Prior to Demonstration 6
- Demo: CLI Launch and Syntax format 7
- Demo: [Check 1] Information Leakage Prevention.....8-9
- Demo: [Check 2] Enforcing HTTPS connection 10-11
- Demo: [Check 3] Manager/Host-Manager Access Controls 12-13
- Demo: [Check 4] Password Encryption 14-15

Analysis and Conclusions

- Analysis Post-Hardening 16
- Project Deliverables 16
- Areas for Future Enhancement 17
- Closing Remarks 17

Project Objectives

The objective of this project was to design and implement a read-only compliance auditing tool for Apache Tomcat servers. The scanner evaluates Tomcat configurations against the CIS Tomcat Benchmark and OWASP Security Misconfiguration guidelines, identifying configurations that could expose the server to security risks. Operating exclusively in read-only mode, the tool ensures that no modifications are made to the target system during the auditing process, maintaining system integrity whilst providing comprehensive security assessments.

All testing was performed within a Kali Linux environment. This choice was driven both by my familiarity with the platform and by the project's conceptual foundation, which draws inspiration from Lynis: a widely adopted security auditing tool for Unix-based systems that assists with compliance testing (including HIPAA, ISO 27001, and PCI DSS standards) and system hardening. The Kali Linux environment provided an appropriate platform for security-focused development and testing, aligning with professional penetration testing and security auditing practices.

The project encompassed several key deliverables: the development of a Python-based scanning tool capable of analysing multiple aspects of Apache Tomcat version 11.0.14 configuration; testing across multiple Tomcat environments representing different security hardening levels; and the generation of compliance reports with clear status indicators (**COMPLIANT**, **PARTIAL**, and **NON-COMPLIANT**). The testing methodology involved identifying compliance and configuration issues within baseline environments, implementing appropriate hardening measures, and subsequently re-running the scanner to validate the effectiveness of security improvements. This iterative approach enabled direct observation of how specific configuration changes enhanced the server's security posture and provided measurable progression from a vulnerable default state to a fully compliant hardened configuration.

Scope and Testing Environment

Compliance checks focus on four principal security domains:

1. Information Leakage Prevention – Mitigating reconnaissance attacks by removing version disclosure in HTTP response headers and error pages.
2. HTTPS Enforcement – Ensuring encrypted communications to protect data in transit through proper SSL/TLS connector configuration and port rerouting.
3. Access Controls for Management Interfaces – Preventing unauthorised administrative access to Manager applications through IP restrictions.
4. Password Encryption – Validating that administrative credentials are protected using SHA-256 hashing rather than plaintext storage in configuration files.

Each check addresses configuration weaknesses frequently exploited in real-world attack scenarios and aligns with professional hardening guidelines and industry best practices, specifically the CIS Apache Tomcat 9 Benchmark v1.2.0 (September 21, 2022). While the benchmark targets Tomcat 9, the security principles remain applicable and relevant to Tomcat 11, as the core configuration architecture and security considerations are consistent across versions.

The scanner delivers formatted output to the console with colour-coded compliance indicators for immediate visual assessment, alongside optional export to JSON or text file formats as specified by the user via command-line arguments.

Problem Statement and the Importance of Secure Configuration

Web application servers such as Apache Tomcat are critical components of enterprise infrastructure, however, default configurations often introduce significant security vulnerabilities that remain unaddressed in production environments. Misconfigurations in areas such as information disclosure, authentication mechanisms, access controls, and communication protocols can expose organisations to substantial risks.

Each of the weaknesses of default Tomcat configuration explored in this report represents a documented attack vector that directly threatens the confidentiality, integrity, or availability of the server and its hosted applications.

By identifying these misconfigurations through automated scanning and demonstrating their remediation through progressive hardening, this project provides a practical, reproducible framework for reducing Tomcat's attack surface.

Documentation and Usability

I implemented professional documentation practises to ensure the program's accessibility and reproducibility. A comprehensive READ ME file accompanies the codebase, as requested in the project outline, providing installation instructions, dependency requirements, usage examples, and troubleshooting guidance. To streamline the installation process, a requirements.txt file is included, allowing all Python dependencies to be installed with a single command (pip install -r requirements.txt). This approach ensures that the scanner can be deployed consistently across different environments without manual dependency resolution.

I also implemented a --help command-line option that displays all available arguments with detailed descriptions. This built-in help functionality ensures users can quickly understand the scanner's capabilities and correct usage patterns without consulting external documentation. The implementation of this --help feature will be demonstrated during the practical assessment.

Scanner Architecture and Functionality
The scanner was developed using Python and structured around a modular architecture that separates concerns into distinct components. This design decision facilitates maintainability, extensibility, and clarity of purpose for each module:

File 1: tomcat_scanner/__init__.py
Makes tomcat_scanner a Python package and defines its public interface.

File 2: tomcat_scanner/config.py
Centralises all configuration values and constants.
- Defines scanner version, timeouts, and user agent strings
- Lists default Tomcat webapps to check (/docs/, /examples/, /manager/, /host-manager/)
- Specifies manager application endpoint paths
- Contains security header definitions with descriptions
- Configures IP filtering rules (allowed/denied IPs) for manager access control validation
- Provides ScannerConfig class for runtime configuration settings

File 3: tomcat_scanner/checks.py
Implements all security check logic via SecurityChecks class.
- **check_information_leakage()** – Detects accessible default webapps, checks for version disclosure in headers and error pages
- **check_https_enforcement()** – Verifies that the target URL uses HTTPS protocol

- **check_manager_access()** – Parses context.xml files to validate RemoteAddrValve configuration and IP-based access controls for manager applications

File 4: tomcat_scanner/reporter.py
Handles all result formatting and output.
- print_results(): Generates colour-coded console output (green= **COMPLIANT**, red= **NON-COMPLIANT**, yellow= **PARTIAL**)
- save_json(): Exports results in JSON format for processing
- save_text(): Creates plain text reports
- Calculates summary statistics (compliance rate, totals)

File 5: tomcat_scanner/scanner.py
Orchestrates the entire scanning workflow.
- Sets up HTTP session with appropriate headers and SSL configuration
- Executes all security checks sequentially via run_scan()

File 6: tomcat-scan.py
Command-line entry point.
- Parses command-line arguments (--target, --verbose, --export)
- Validates user input (ensures URLs start with http:// or https://)
- Creates scanner instance and executes scan

This modular approach offers several practical advantages. From a development perspective, it simplified debugging and maintenance by isolating functionality; issues within specific checks could be readily identified and resolved without navigating extensive monolithic code. Furthermore, the architecture supports extensibility, new security checks can be incorporated by implementing additional methods within checks.py without requiring substantial refactoring of the core scanner logic.

The subsequent sections of this report demonstrate the identification of specific security misconfigurations, explain their potential impact, detail the hardening measures implemented, and present validation results confirming improved compliance status.

Environment Preparation Prior to Demonstration

Before running the Python script to demonstrate Apache Tomcat configuration compliance, I prepared the environment to ensure it followed standard security hardening practices. These preliminary steps were necessary to establish a secure baseline and prevent false positives when verifying configuration compliance.

Establishing Secure File Ownership and Permissions

I configured Tomcat with the recommended file ownership and permission structure:

- Owner: tomcat
- Group: tomcat
- Permissions:
    - bin/ and conf/ directories → 750
    - Configuration files restricted to the Tomcat service account only

These steps ensure that only the Tomcat service account can access or modify sensitive configuration files. This protects against:

- Unauthorised configuration tampering - Only Tomcat itself (not shell users) can read the configuration and binaries.
- Privilege escalation risks - Prevents a compromised low-privileged user from altering startup scripts or server.xml to gain further access.
- Accidental modification - Restricting world access reduces the attack surface and enforces the principle of least privilege.

This configuration aligns with common security guidance such as CIS Benchmarks and OWASP recommendations for securing Apache Tomcat.

Demo: CLI Launch and Program Execution

When initially launching the Python compliance scanner, I deliberately tested it with an incorrect command format to verify its error handling capabilities.

```
└─$ python3 tomcat-scan.py
usage: tomcat-scan.py [-h] --target TARGET [-v] [--export FILE] [--version]
tomcat-scan.py: error: the following arguments are required: --target
```

The tool provides helpful user guidance by displaying the correct command-line syntax whenever an incomplete or invalid command is entered. For example, when the URL parameter is omitted (as shown in the screenshot above), the program prompts the user with the proper usage format. Additionally, the scanner supports exporting results via the --export option, allowing users to save findings for later review. The program also includes comprehensive contextual help accessible through the -h or --help flag (screenshot below).

```
└─# python3 tomcat-scan.py --help
usage: tomcat-scan.py [-h] --target TARGET [-v] [--export FILE] [--version]

Apache Tomcat Security Compliance Scanner

options:
  -h, --help        show this help message and exit
  --target TARGET   Target Tomcat server URL (e.g., http://192.168.1.100:8080)
  -v, --verbose     Enable verbose output with details and recommendations
  --export FILE     Export results to file (e.g., results.json)
  --version         show program's version number and exit

Examples:
  python3 tomcat-scan.py --target http://192.168.1.100:8080
```

These tests confirmed that the scanner functions correctly and provides sufficient guidance for users unfamiliar with the tool to understand its operation.

To begin the demonstration, I executed the compliance scanner (with the correct syntax) against the default Tomcat installation version. As expected, the initial run flagged every security check as **NON-COMPLIANT**, confirming that the server had not yet undergone any hardening/configuration tightening.

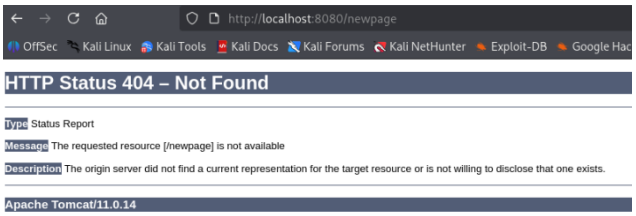
```
TOMCAT SECURITY COMPLIANCE CHECK RESULTS
[CHECK] Information Leakage Prevention ..................... NON-COMPLIANT
[CHECK] HTTPS Enforcement .................................. NON-COMPLIANT
[CHECK] Manager/Host-Manager Access Controls ............... NON-COMPLIANT
[CHECK] Password Encryption (tomcat-users.xml) ............ NON-COMPLIANT
SUMMARY
Total Checks:         4
Compliant:            0
Partially Compliant:  0
Non-Compliant:        4
Unknown:              0

Compliance Score:     0.0%
```

I chose to generate a text report using --export results.txt in the command line to view the reasons behind each **NON-COMPLIANT** finding. From this point onward, I approached the hardening process methodically, addressing each compliance check one at a time, applying the recommended security fixes, and then re-running the scanner to verify that the changes were effective before proceeding to the next control.

Demo: [Check 1] Information leakage

The first compliance check which I address in this report is the Information Leakage prevention.

During testing in class, we observed that when requesting a non-existent URL (e.g. http://localhost:8080/newpage), Tomcat returned a 404 Not Found page that revealed sensitive server details. Specifically, the response exposed the full Tomcat version string, as shown in the screenshot below.



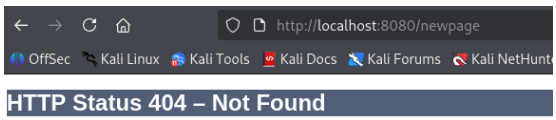
This leakage occurs through Tomcat's default error pages and is considered a security risk because attackers can use precise version information to target known vulnerabilities.

Hardening — Hiding error reports and server information

I inspected server.xml and found that no error-reporting valve had been configured. To prevent version disclosure and remove verbose error details, I added the following valve:

```
      <Host name="localhost"  appBase="webapps"
            unpackWARs="true" autoDeploy="true">

        <!-- SingleSignOn valve, share authentication between web applications
             Documentation at: /docs/config/valve.html -->
        <!--
        <Valve className="org.apache.catalina.authenticator.SingleSignOn" />
        -->

        <!-- Access log processes all example.
             Documentation at: /docs/config/valve.html
             Note: The pattern used is equivalent to using pattern="common" -->
        <Valve className="org.apache.catalina.valves.AccessLogValve" directory="logs"
               prefix="localhost_access_log" suffix=".txt"
               pattern="%h %l %u %t &quot;%r&quot; %s %b" />
        <Valve className="org.apache.catalina.valves.ErrorReportValve"
               showReport="false"
               showServerInfo="false" />
      </Host>
```

After applying this change and restarting Tomcat, error pages no longer displayed server version information, confirming that the leakage was mitigated for 404 Not Found pages.



Hardening — Removing default web applications

Next, I removed the default Tomcat webapps from /opt/tomcat/webapps, including /docs and /examples.

Why this improves compliance:

- These applications often contain documentation that exposes Tomcat version numbers.
- Sample applications can reveal internal implementations and increase the attack surface.
- Security baselines (e.g., CIS Benchmarks, OWASP) recommend removing all default/demo apps from production environments.

After completing both hardening actions, I re-ran the compliance scanner, and the result for the Information Compliance prevention changed from **NON-COMPLIANT** to **COMPLIANT**.

```
TOMCAT SECURITY COMPLIANCE CHECK RESULTS
================================================================================
[CHECK] Information Leakage Prevention ..................... COMPLIANT
[CHECK] HTTPS Enforcement ................................. NON-COMPLIANT
[CHECK] Manager/Host-Manager Access Controls .............. NON-COMPLIANT
[CHECK] Password Encryption (tomcat-users.xml) ............ NON-COMPLIANT
================================================================================
SUMMARY
================================================================================
Total Checks:        4
Compliant:           1
Partially Compliant: 0
Non-Compliant:       3
Unknown:             0

Compliance Score:    25.0%
================================================================================
```

This confirmed that the information leakage issue had been successfully resolved and that the Tomcat configuration now adheres to recommended security practices.

Demo: [Check 2] Enforcing HTTPS connections

The second compliance control evaluated relates to enforcing secure HTTPS connections. By default, Tomcat operates using an unencrypted HTTP connector, which naturally results in a **NON-COMPLIANT** status when launching my Python program. Without HTTPS, all data transmitted between the client and server, including sensitive information such as usernames, passwords, and session tokens, travels in plaintext over the network. This makes the application vulnerable to man-in-the-middle (MITM) attacks, eavesdropping, and session hijacking. To meet the security requirement, I enabled HTTPS, generated an SSL certificate, and configured Tomcat to redirect all HTTP traffic to a secure port (not the default 8080 port).

1. Generating a Self-Signed Certificate

To activate HTTPS, I first created a self-signed RSA certificate (using the password **dstiproject**) and stored it in a new keystore (**tomcat.keystore**). While self-signed certificates are not suitable for production, they can be used in testing and demonstration to verify that HTTPS is functioning correctly and that Tomcat properly handles SSL connections. The aim at this stage was simply to confirm that Tomcat could successfully load the keystore and perform the SSL handshake.

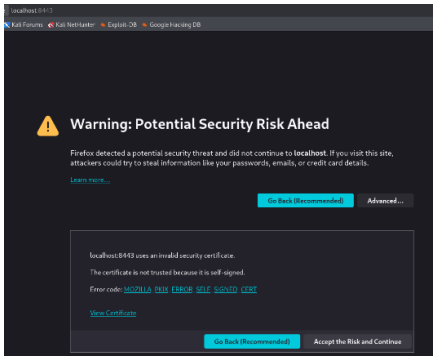
```
tomcat@kali:~$ mkdir -p /opt/tomcat/conf/ssl
cd /opt/tomcat/conf/ssl
tomcat@kali:~/conf/ssl$ keytool -genkey -alias tomcat -keyalg RSA -keysize 2048 -keystore tomcat.keystore -validity 365
Enter keystore password:
Re-enter new password:
```

After generating the certificate, I added an HTTPS connector to the server.xml file and pointed it to **tomcat.keystore**. This configuration enabled SSL/TLS on port **8443** and ensured that Tomcat used the correct certificate during HTTPS negotiations.

```
-->

<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
           maxThreads="150" SSLEnabled="true">
    <SSLHostConfig>
        <Certificate certificateKeystoreFile="conf/ssl/tomcat.keystore"
                     certificateKeystorePassword="dstiproject" type="RSA" />
    </SSLHostConfig>
</Connector>
```

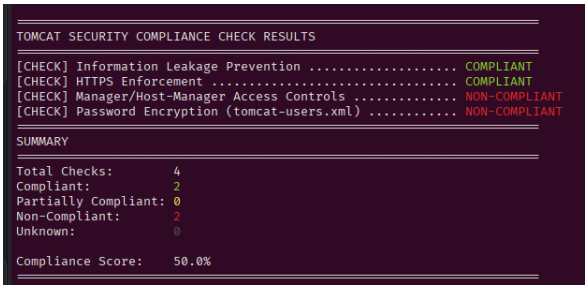
To complete the setup, I modified the existing HTTP connector to include a redirect rule. This ensures that any request to http://localhost:8080 is automatically forwarded to the secure https://localhost:8443 endpoint, guaranteeing that all traffic is served over HTTPS.

Now, when re-starting Apache Tomcat via the ./startup.sh command, and accessing http://localhost:8080, the browser correctly redirected to https://localhost:8443, however, as expected with a self-signed certificate, the browser displayed a security warning.



Examining the advanced details confirmed that the warning occurred because the certificate was locally issued rather than signed by a trusted certificate authority. Although, as mentioned, this is not suitable for production, it demonstrates that HTTPS is correctly enabled via the SSL connector, the RSA keystore is valid, and HTTPS redirection is functioning correctly.

With HTTPS enabled and functional, I re-launched the Python compliance scanner using *https* instead of *http* and port *8443* instead of port *8080* in the command line. The screenshot below shows that now, the HTTPS Enforcement check satisfies the compliance requirement for this project.



Areas for Future Improvement

Additional HTTPS-related hardening measures could be implemented in the check.py script to enforce stricter compliance. Examples of such practices include:

- X-Frame-Options → prevents clickjacking
- X-Content-Type-Options → prevents MIME-type sniffing
- Strict-Transport-Security (HSTS) → enforces long-term HTTPS

While these headers are recommended best practices for production environments, they were not included in this stage in order to keep the report concise. They can be considered as potential enhancements for future improvements.

Demo: [Check 3] Manager/Host-Manager Accessibility

The Tomcat Manager UI is a critical administrative interface and represents a high-risk attack surface. If left unprotected, it can be exploited to:

- Deploy malicious applications containing backdoors or malware
- Execute arbitrary code on the server
- Extract sensitive configuration data and credentials
- Gain full control of the Tomcat instance

In short, an exposed Manager UI can allow an attacker to fully compromise the server, and when launching my program, the check compliance results indicate that this Manager/Host Manger access control check is **NON-COMPLIANT**. The audit export file indicated that this was due to the fact that No IP filtering was configured. The program works here by scanning the configuatation files, specficially in the context.xml file in manager directory, to find RemoteAddrValve value, which would then allow or deny specific IP addresses. As the RemoteAddrValve was not found in any of the context.xml files, this causes the check to trigger **NON-COMPLIANT** result.

Best practise standards indicate that access should be restricted to Localhost **(127.0.0.1)** for local administration and Specific trusted network ranges (e.g., 192.168.1.0/24) for authorised administrators This approach ensures that the Manager UI cannot be accessed from external or untrusted IP addresses, significantly reducing the attack surface.

Hardening

I implemented IP-based restrictions by adding a RemoteAddrValve to the webapps/manager/META-INF/context.xml file. The configuration restricts access to 127.0.0.1 only, as shown in the screenshot below:

```
<Context antiResourceLocking="false" privileged="true">
  <Valve  className="org.apache.catalina.valves.RemoteAddrValve"
          allow="127\.0\.0\.1|::1" />
</Context>
```

I verified my connecting IP address by examining the Tomcat access logs using *tail -f*, which revealed that my browser was connecting from 0:0:0:0:0:0:0:1 (expanded form of IPv6 localhost). My initial context.xml configuration only permitted 127\.0\.0\.1 (IPv4 localhost) and ::1 (IPv6 localhost short form). To ensure successful access to the Manager application, I modified the configuration to also include 0:0:0:0:0:0:0:1. This adjustment was necessary because browsers and systems may use either the short or expanded IPv6 format when connecting to localhost, failing to account for both

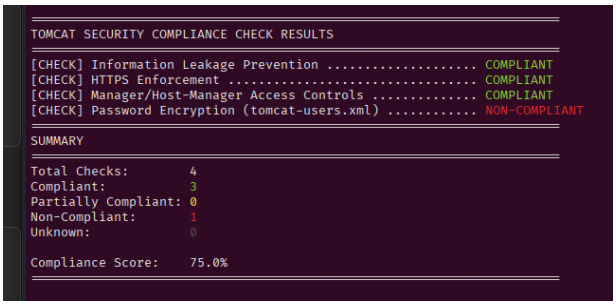
representations would result in legitimate local access being blocked by the RemoteAddrValve filter.

If the host-manager webapp was present in my environment, the same configuration would be applied in the webapps/host-manager/META-INF/context.xml directory. However, this directory did not exist in my Tomcat environment, so I applied the IP restriction only to the Manager context, reducing the attack surface and simplifying management.

Another way to harden this configation: Login credentials

At this stage, only IP-based restrictions are applied. Login credentials for the Manager UI (defined in tomcat-users.xml) are not addressed in this check; they are covered in a subsequent compliance step focusing on **encrypted credentials and secure user configuration**. For the purposes of this check, enforcing IP restrictions alone is sufficient to limit access to trusted networks and reduce exposure.

After applying the IP restrictions and restarting Tomcat, I re-ran the compliance scanner. The Manager UI check now passed and was marked **COMPLIANT**, confirming that access is correctly limited to trusted IPs.



Demo: [Check 4] Password Encryption

My scanner marked the password encryption check as **NON-COMPLIANT** because: (a) it did not locate any CredentialHandler configuration in server.xml, and (b) the password stored in tomcat-users.xml was in plaintext. I chose to configure my *check.py* script in my programme to search for CredentialHandler configuration in server.xml because this is the mechanism used to enforce password hashing, allowing Tomcat to interpret passwords as hashed values. Without the CredentialHandler, Tomcat cannot compare the user's input during the login process against stored hashes - it would attempt to compare plaintext-to-plaintext, making hashing pointless.

Security Issue

Storing plaintext passwords in tomcat-users.xml represents a critical security risk, as anyone with read access to the configuration file can view user credentials directly. This vulnerability exposes the system to credential theft and unauthorised access.

Compliance standards and best practice guidelines require that passwords must be hashed to prevent exposure in the event of file compromise.

Hardening: **Password Hash Generation**
I generated a hashed version of the chosen password (*dsti_tomcat_password*) using Tomcat's SHA-256 hashing command, which produced the hash shown in the screenshot below.

```
dsti_tomcat_password:d0d13a176bc6c3de072be9355687cbc1181cd2b079da40554f1ea5a843bf3183$1$00212c1d98f1fd1f97e7911ad19b1243938ccd1295c6b4b519507fe79a0207c6
tomcat@kali:~/bin$
```

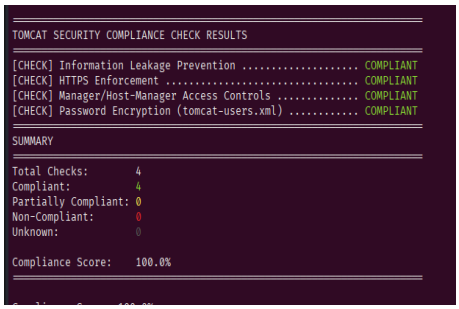
Hardening: Configuration Update
I replaced the plaintext password (*dsti_tomcat_password*) in tomcat-users.xml with the generated hash for the admin user. I then configured the CredentialHandler in server.xml to ensure Tomcat correctly interprets passwords entered as plaintext via the web interface, mapping them to their hashed equivalents stored in tomcat-users.xml.

```
        via a brute-force attack -->
    <Realm  className="org.apache.catalina.realm.LockOutRealm">
      <Realm  className="org.apache.catalina.realm.UserDatabaseRealm"  resourceName="UserDatabase">
            <CredentialHandler  className="org.apache.catalina.realm.MessageDigestCredentialHandler"  algorithm="SHA-256"  />
      </Realm>
      </Realm>
```

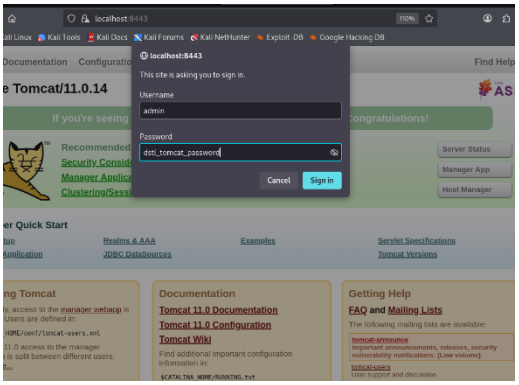
Hardening: File Permissions
I verified that all configuration files (especially server.xml and tomcat-users.xml) had permissions restricted to the admin user (600) to meet best practice recommendations.

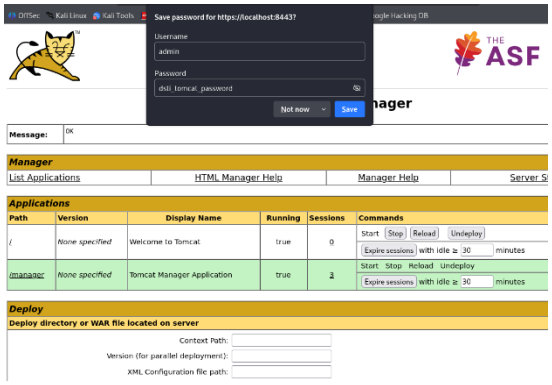
Verification

I re-ran the compliance scanner, which verified the hash against the CredentialHandler configuration and reported **COMPLIANT**.



I then confirmed that authentication still functioned correctly despite the credential changes by logging into the Tomcat Manager web application using the original plaintext password (which is hashed internally during the authentication process). Access was successfully granted, as shown in the screenshots below.





By enforcing hashed passwords and securing configuration file permissions, I mitigated the risk of credential leakage, ensured compliance with best practice standards, and maintained operational access to Tomcat services.

Analysis Post-Hardening

This project successfully demonstrated a systematic approach to Apache Tomcat security compliance through automated validation and progressive hardening. The Python-based compliance checking tool, developed specifically for this assessment, proved effective in identifying security gaps and verifying remediation measures across four critical security domains.

The audit results clearly illustrate the stark contrast in security posture between a default Tomcat installation and a properly hardened configuration. The initial compliance scan revealed that all four checks returned **NON-COMPLIANT** status, yielding a 0% compliance score. Through methodical hardening procedures demonstrated in each section, the final scan achieved **100% compliance** across all validated checks.

Each check addressed a specific vulnerability class with security implications:

- **Information Leakage Prevention** – Mitigated reconnaissance attacks by suppressing server version disclosure, reducing an attacker's ability to identify known vulnerabilities.
- **HTTPS Enforcement** – Ensured encrypted communications to protect sensitive data in transit, preventing man-in-the-middle attacks and credential interception.
- **Access Controls for manager interfaces** – Prevented unauthorised administrative access to the Manager application through IP restriciton/white-listing, and reduced the attack surface by not including the unused Host Manager application.
- **Credential Security** – Implemented password encryption using SHA-256 hashing to protect administrative credentials from exposure in plaintext configuration files.

The progressive hardening demonstrations illustrated not only *what* to change, but *why* each modification strengthens the security posture and *how* it addresses specific threat vectors.

Project Deliverables

This project has fully met all requirements outlined in the project specification:
- Complete Python compliance checking script deployed on a Linux environment
- Validated against Apache Tomcat 11.0.14 installation
- Comprehensive READ ME documentation accompanying the codebase
- Built-in --help functionality in the CLI interface providing clear execution instructions
- Summary table documenting all checks with **NON-COMPLIANT**, **PARTIAL**, and **COMPLIANT** grading
- Colour-coded visual indicators for rapid interpretation of compliance status
- Compliance scoring mechanism providing clear percentage-based metrics (0% to 100%)

- In-depth analysis of detected misconfigurations with actionable recommendations
- Supporting evidence including screenshots and command outputs validating all findings

Areas for Future Enhancement

While this project successfully addressed the core security requirements, several opportunities exist for extending the compliance framework.

As mentionned previously on page, additional HTTPS-related hardening measures could be integrated into the check.py script to enforce more comprehensive web security controls:

- **X-Frame-Options** – Prevents clickjacking attacks by controlling whether the application can be embedded in frames
- **X-Content-Type-Options** – Prevents MIME-type sniffing vulnerabilities by forcing browsers to respect declared content types
- **Strict-Transport-Security (HSTS)** – Enforces long-term HTTPS usage by instructing browsers to always use secure connections

Closing Remarks

This project demonstrates that security compliance is not a binary state but rather a spectrum of configurations with measurable outcomes. The journey from 0% to 100% compliance, achieved through systematic validation and remediation, provides a reproducible methodology for securing Apache Tomcat deployments in production environments. While the four checks implemented provide a solid foundation, they represent a starting point rather than a comprehensive security solution.